

**Implementación de un pipeline de observabilidad con OpenTelemetry: arquitectura,
correlación cross-signal y análisis de overhead**

Fredy Orlando Pulido Quintero

Myriam Andrea Martínez Fontecha

Juan Francisco Javier Pérez Rivero

Nicolás Felipe Torres Amaya

Maestría en Arquitectura de Software

MASS – OBAP20264: Observabilidad en Ambientes Productivos

Maria Fernanda Ochoa Paipilla

24 de agosto de 2026

Implementación de un pipeline de observabilidad con OpenTelemetry

Introducción

La observabilidad no verifica umbrales conocidos, sino que permite responder preguntas no anticipadas (Majors et al., 2022). El objetivo de este laboratorio no es recolectar telemetría, sino demostrar que las tres señales pueden recorrerse entre sí para investigar una petición concreta, y cuantificar el costo de esa capacidad. El sistema modela un checkout con dos servicios —el problema relevante del trazado distribuido es el salto entre procesos— y una base de datos PostgreSQL.

Arquitectura de la Solución

La aplicación exporta telemetría por OTLP/gRPC hacia un OpenTelemetry Collector que la distribuye a tres backends especializados. Ese desacople es la decisión arquitectónica central: la aplicación conoce una sola dirección y el destino se resuelve en configuración.

Tabla 1

Componentes del pipeline y responsabilidad de cada uno

Componente	Señal	Función
OTel SDK (Python)	Las tres	Instrumentación automática y manual en el proceso
OTel Collector	Las tres	Recepción, límite de memoria, etiquetado y reenvío
Jaeger	Trazas	Almacenamiento y visualización en cascada
Prometheus	Métricas	Series temporales y exemplars
Loki	Logs	Indexación por etiquetas, no por texto

Nota. El Collector aplica los procesadores en el orden `memory_limiter` → `resource` → `resourcedetection` → `filter` → `batch`.

Decisiones de Diseño

Instrumentación automática y manual. La automática se activa con el comando `opentelemetry-instrument`, sin envolver el código: cubre cada petición HTTP, cada consulta SQL y la propagación de contexto W3C (W3C, 2021). Su límite es que desconoce el dominio — observa una sentencia `UPDATE`, no una reserva de inventario—, por lo que se añadieron tres spans de negocio.

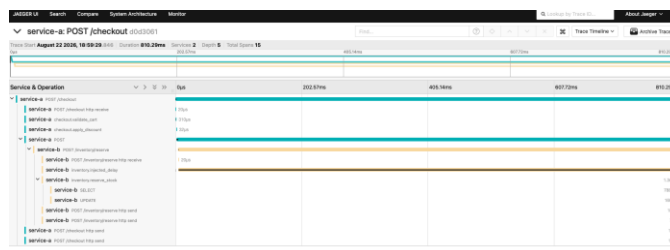
Orden de los procesadores y cardinalidad. El limitador de memoria va primero porque su función es rechazar datos bajo presión: situado tras el agrupamiento ya habría consumido la memoria que intenta proteger, y un Collector que termina por falta de memoria deja al sistema ciego durante un incidente. Los identificadores de carrito, por su parte, se registran como atributos de span y nunca como etiquetas de métrica: una etiqueta de cardinalidad alta genera una serie temporal por valor y agota el sistema de métricas.

Correlación Cross-Signal

La correlación se verificó sobre una misma petición. Prometheus almacenó un exemplar de 809 ms que referencia el identificador de traza; Jaeger contiene esa traza con 15 spans y 810,29 ms repartidos entre ambos servicios; y Loki devuelve siete líneas de registro de los dos servicios al filtrar por ese valor. La coincidencia entre el exemplar y la duración real confirma que la cadena opera de extremo a extremo. Exige cinco eslabones correctamente configurados, y cada uno falla en silencio.

Figura 1

Traza distribuida con spans automáticos y de negocio en un mismo árbol



Nota. Los spans checkout.validate_cart e inventory.reserve_stock son manuales; SELECT y UPDATE los genera la instrumentación automática de pycopg2 y aparecen anidados bajo el span de negocio.

Análisis de Overhead

Se compararon dos escenarios con k6 —SDK desactivado frente a instrumentación completa— bajo 50 usuarios virtuales durante 420 segundos, con tres corridas por escenario y descarte de la primera como calentamiento. Las seis resultaron válidas, con cero errores inesperados.

Tabla 2

Latencia y rendimiento por escenario

Métrica	Sin instrumentar	Instrumentado	Diferencia
Latencia p50	135,5 ms	167,5 ms	+32,0 ms (23,6 %)
Latencia p95	199,5 ms	257,0 ms	+57,5 ms (28,8 %)
Latencia p99	245,5 ms	329,0 ms	+83,5 ms (34,0 %)
Rendimiento	336,6 sol/s	270,1 sol/s	−19,7 %

Nota. Media de dos corridas. La desviación entre corridas fue de 5,3 % y 1,7 % sobre el p95, frente a una diferencia entre escenarios de 28,8 %.

Los cuatro valores de la Tabla 2 no son hallazgos independientes. La prueba opera en lazo cerrado con concurrencia fija, régimen donde rige la ley de Little (Little, 1961): el tiempo de respuesta equivale al número de usuarios dividido entre el rendimiento. Esa relación predice un aumento del 24,6 % y se midió 23,6 %, lo que confirma que el alza de latencia y la caída de rendimiento son el mismo fenómeno.

El consumo de procesador exigió otro tratamiento. En términos absolutos resultaba engañoso: el servicio A registró menos uso al estar instrumentado, pero no es un ahorro sino

menos peticiones atendidas, ya que ambos escenarios operan saturados. Normalizarlo por el rendimiento entrega la magnitud comparable.

Tabla 3

Consumo de procesador por petición

Capa	Sin instrumentar	Instrumentado	Diferencia
Aplicación y base de datos	0,7937	0,9452	+19,1 %
Backends de telemetría	0,0042	0,0797	—
Total del sistema	0,7978	1,0249	+28,5 %

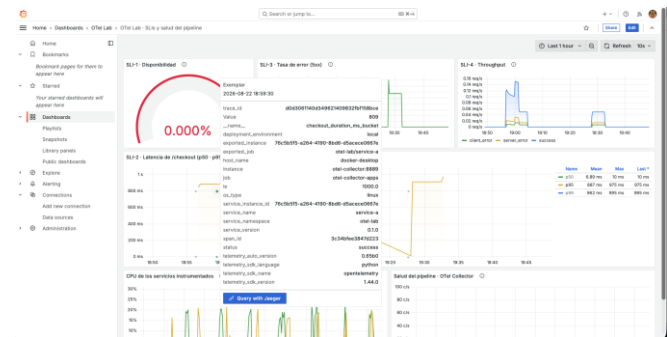
Nota. Unidad: porcentaje de procesador dividido entre solicitudes por segundo. Del sobre costo total, 67 % corresponde a la aplicación y 33 % a los backends. El consumo adicional de memoria fue de 4,6 MB y 4,3 MB por servicio.

El costo transferible es el 28,5 % de procesador por petición, no el 34 % del percentil 99: este último depende de que el servicio B operara al 98,8 % de procesador. Con holgura, el mismo incremento de tiempo de servicio produce un alza de latencia mucho menor, al no existir cola que la amplifique.

La primera ejecución resultó inválida por miles de errores, causados por un conjunto de conexiones que admitía cinco frente a los cuarenta hilos con que el servidor atiende funciones síncronas. Corregido el defecto, la medición se repitió. Una prueba de humo secuencial jamás lo habría detectado, lo que respalda medir bajo carga y no solo comprobar que los extremos responden.

Figura 2

Exemplar que enlaza un punto de la métrica de latencia con su traza



Nota. El cuadro emergente muestra el identificador de traza asociado al valor de 809 ms y el enlace directo hacia Jaeger.

Despliegue en la Nube

Elección del proveedor.

Se eligió Google Cloud Platform por una razón económica, dado los niveles y servicios gratuitos.

Portabilidad sin cambios en el código.

La aplicación se desplegó sin modificar una sola línea. Entre la configuración local del Collector y la de la nube, los receptores, los procesadores y las tres canalizaciones son idénticos; lo único que cambia son los exportadores.

Tabla 4

Equivalencia de backends entre el entorno local y la nube

Señal	Entorno local	Google Cloud
Trazas	Jaeger	Cloud Trace
Métricas	Prometheus	Managed Service for Prometheus
Registros	Loki	Cloud Logging

Nota. Los backends no se trasladaron a máquinas virtuales, sino que se sustituyeron por los servicios gestionados equivalentes, lo que mantiene el despliegue dentro del nivel gratuito.

Arquitectura del despliegue.

Cada servicio se ejecuta en Cloud Run con su propio Collector como contenedor adjunto. Al compartir el espacio de red de la instancia, la aplicación exporta a la dirección local, el Collector no queda expuesto a internet y se evita un conector de red privada, que sí tendría costo. El servicio de inventario incorpora además PostgreSQL como tercer contenedor adjunto, lo que conserva intactos los spans de base de datos: migrar a un motor embebido habría cambiado la instrumentación que sustenta el primer criterio. Todo se define en Terraform, con la región fijada por una validación que rechaza cualquier otra, pues solo una ofrece nivel gratuito.

Un hallazgo del despliegue.

La aplicación respondía correctamente y sus registros llegaban a Cloud Logging, pero Cloud Trace no recibía ninguna traza y ningún componente reportaba error. La causa: Cloud Run asigna procesador únicamente mientras se atiende una petición, y al enviar la respuesta lo congela, de modo que el hilo que exporta los spans en segundo plano nunca llegaba a ejecutarse. Los datos se acumulaban en la cola y se perdían en silencio. Un exportador de depuración temporal demostró que los spans sí alcanzaban al Collector, lo que descartó al SDK; variar el ritmo del tráfico resultó decisivo.

Seis peticiones seguidas entregaron 8 spans de los 42 esperados, mientras que esas mismas seis, espaciadas cinco segundos, entregaron las seis trazas completas. La corrección consistió en asignar procesador de forma permanente y acortar los lotes a un segundo, tanto en el SDK como en el Collector.

El valor de este hallazgo excede al laboratorio. Una arquitectura de telemetría que funciona sin fallos en un contenedor de larga vida puede perder datos en una plataforma sin

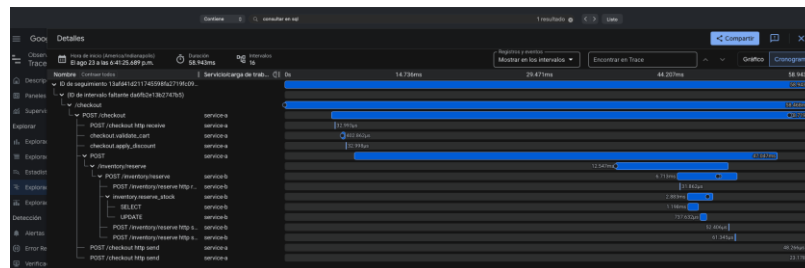
servidor por un motivo que no aparece en ningún registro. Es exactamente el tipo de diferencia entre entornos que justifica desplegar en la nube en lugar de dar por válido lo que funciona en local.

Verificación.

Los tres pilares se comprobaron sobre una misma petición: Cloud Trace muestra la traza con dieciséis spans repartidos entre los dos servicios, Cloud Logging devuelve las líneas de ambos al filtrar por ese identificador, y Managed Prometheus conserva las métricas con sus etiquetas. Los nombres de las métricas son idénticos a los locales, así que las consultas del tablero anterior funcionan en ambos entornos sin modificación.

Figura 3

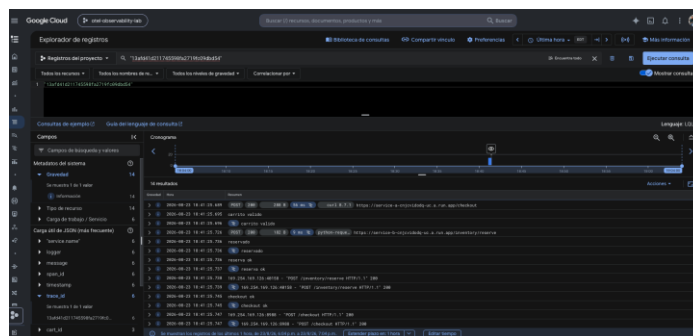
Traza distribuida en Cloud Trace



Nota. Los spans `checkout.validate_cart`, `inventory.reserve_stock`, `SELECT` y `UPDATE` aparecen igual que en Jaeger, lo que confirma que la instrumentación viajó sin cambios.

Figura 4

Registros de esa misma traza en Cloud Logging



Nota. La consulta filtra por el identificador de traza y devuelve líneas de los dos servicios. El panel lateral muestra `cart_id` y `trace_id` como campos indexados.

Limitaciones.

Bajo ráfagas sostenidas la plataforma sigue perdiendo lotes, porque las instancias se crean y destruyen con rapidez; capturar evidencia exige espaciar el tráfico. En el entorno local, con los mismos servicios y cincuenta usuarios concurrentes, la entrega fue completa. Y al desplegarse en un solo proveedor, la portabilidad queda demostrada por diseño y no por ejecución.

Medición del overhead en la nube.

El análisis de overhead se repitió sobre Cloud Run con un arnés distinto al local, porque medir desde un equipo remoto introduce tres distorsiones. La latencia de red hasta la región es de unos 182 milisegundos, seis veces el efecto que se pretende medir; por ello la latencia no se cronometra desde el generador de carga, sino que se leen las métricas que la plataforma registra en su propio borde. El autoescalado se neutraliza fijando una sola instancia durante la medición. Y la pérdida de telemetría bajo ráfagas, descrita antes, produciría un sesgo a favor de la instrumentación —al perder datos el servicio realiza menos trabajo y parece más rápido—, de modo que el análisis publica el número de peticiones atendidas por escenario para que ese sesgo sea verificable.

Tabla 5*Overhead medido en Cloud Run*

Métrica	Sin instrumentar	Instrumentado	Diferencia
Latencia p50	74,0 ms	90,0 ms	+15,9 ms (21,6 %)
Latencia p95	137,7 ms	158,6 ms	+21,0 ms (15,2 %)
Memoria media	29,3 %	32,2 %	+2,9 pp (10,0 %)
Rendimiento	47,1 sol/s	39,2 sol/s	-16,8 %

Nota. Seis corridas, todas válidas, sin errores inesperados. Perfil de 15 usuarios virtuales, más corto que el local porque la red limita la carga que puede generarse de forma remota.

Ambos entornos coinciden.

El resultado más valioso del ejercicio no es la cifra en la nube, sino su coincidencia con la del entorno local.

Tabla 6*Comparación del overhead entre los dos entornos*

Métrica	Entorno local	Cloud Run
Latencia p50	+23,6 %	+21,6 %
Rendimiento	-19,7 %	-16,8 %
Memoria	+7 %	+10 %

Nota. Hardware, sistema operativo y perfil de carga distintos.

Que dos entornos tan diferentes arrojen la misma magnitud constituye la mejor validación disponible del resultado: el sobre costo de instrumentar es una propiedad del SDK y no un artefacto del equipo de pruebas.

El percentil 99 es la única métrica que se aparta, con un incremento del 4,8 % frente al 34 % local, y la desviación confirma la interpretación anterior en lugar de contradecirla: en la nube el sistema operaba al 18 % de procesador y no saturado, de modo que no existía cola que amplificara el extremo de la distribución. El efecto sobre el percentil 99 depende de la utilización; el costo de servicio, no.

Conclusiones y Recomendaciones

La instrumentación completa costó 28,5 % de procesador por petición y 4,5 MB de memoria por servicio, a cambio de poder investigar cualquier petición individual a través de las tres señales. El costo de memoria es despreciable; el de procesador justifica muestreo en alto volumen.

La aplicación no conoce su destino y la migración se resuelve en la configuración del Collector, no en el código.

La distribución del sobrecosto orienta esa estrategia en sentido contrario al habitual: como dos tercios se consumen dentro de la aplicación, el muestreo en el Collector solo reduciría el tercio restante, pues el gasto ya ocurrió antes de que el Collector recibiera los datos. La palanca efectiva es el muestreo en origen, que evita crear el span; su contrapartida es la pérdida de trazas de error, ya que la decisión se toma al inicio, y se mitiga conservando métricas y registros íntegros, señales mucho más económicas (Beyer et al., 2018). Se recomienda muestreo íntegro en servicios críticos o de bajo volumen, y entre 10 % y 20 % en origen para servicios de alto volumen sin holgura de procesador.

Referencias

- Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (2016). Site reliability engineering: How Google runs production systems. O'Reilly Media.
- Beyer, B., Murphy, N. R., Rensin, D. K., Kawahara, K., & Thorne, S. (2018). The site reliability workbook: Practical ways to implement SRE. O'Reilly Media.
- Little, J. D. C. (1961). A proof for the queuing formula: $L = \lambda W$. Operations Research, 9(3), 383–387. <https://doi.org/10.1287/opre.9.3.383>
- Majors, C., Fong-Jones, L., & Miranda, G. (2022). Observability engineering: Achieving production excellence. O'Reilly Media.
- OpenTelemetry Authors. (2025). OpenTelemetry documentation. Cloud Native Computing Foundation. <https://opentelemetry.io/docs/>
- Sigelman, B. H., Barroso, L. A., Burrows, M., Stephenson, P., Plakal, M., Beaver, D., Jaspan, S., & Shanbhag, C. (2010). Dapper, a large-scale distributed systems tracing infrastructure (Technical Report dapper-2010-1). Google.
- World Wide Web Consortium. (2021). Trace context (W3C Recommendation). <https://www.w3.org/TR/trace-context/>